

CH03 文件系统原理和权限模型

考试复习笔记

来源：开源操作系统课件（47 页） 日期：2026 年 6 月 23 日

1 文件系统原理 [p.5–21]

1.1 常见 Linux 文件系统 [p.6–7]

文件系统	特点
ext2	无日志，性能较好，崩溃后恢复慢，适合小分区/USB
ext3	ext2 + 日志功能，可靠性大幅提升
ext4	支持超大分区 (1EB)/单文件 (16TB)，延迟分配，快速 fsck，大多数发行版默认
XFS	SGI 开发，高性能日志 FS，擅长高并发 I/O， RHEL/CentOS 7+ 默认，动态 inode 分配
Btrfs	写时复制 (CoW)，支持快照/子卷，内置 RAID(0/1/10)，透明压缩节省 30–50% 空间
vfat/FAT32	U 盘通用，不支持 Linux 权限与软链接，兼容性最好
NTFS	Windows FS，Linux 通过 NTFS-3G 读写
ZFS	企业级高级 FS，CDDL 许可证限制

1.2 文件系统结构组成（ext 索引式） [p.8]

- 1. 引导块 (Boot Block): 分区最前端，存放引导程序
- 2. 超级块 (Super Block): 文件系统元数据（总块数、空闲块、inode 总数、块大小、挂载时间），损坏可能导致无法挂载
- 3. inode 位图: 标记 inode 占用/空闲状态
- 4. 数据块位图: 标记数据块占用/空闲状态
- 5. inode 表: 存放所有 inode 条目（元数据 + 数据块指针）
- 6. 数据块 (Data Blocks): 真正存储文件内容

1.3 文件系统相关命令 [p.9]

```
df -Th          # 查看文件系统类型和挂载信息
lsblk -f        # 查看磁盘分区和类型
```

```
fdisk -l /dev/sda      # 详细查看分区信息
dumpe2fs /dev/sda1    # 查看超级块信息
mkfs.ext4 /dev/sdb1    # 创建 ext4 文件系统
mkfs.xfs /dev/sdc1     # 创建 XFS 文件系统
```

1.4 inode 详解 [p.10–15]

定义：inode (index node) 是 Linux 文件系统核心概念。每个文件/目录对应唯一 inode，保存文件所有元数据，但不存放文件名和文件内容 [p.10]。

文件名存在目录文件内，目录本质是” 文件名 → inode” 映射表。

1.4.1 inode 存储的信息 [p.12]

属性	描述
文件类型	普通文件、目录、软链接、设备文件、管道、套接字
访问权限	rw-rw-rwx (9 个权限位)
所有者	所有者 UID、所属组 GID
大小	文件字节数
时间戳	atime, mtime, ctime 三大时间戳
硬链接计数	指向该 inode 的目录项数量
数据块指针	直接指针、间接指针、二次间接指针、三次间接指针

1.4.2 查看 inode 命令 [p.13]

```
ls -l          # 简单查看文件元数据
ls -li file.txt # 查看 inode 编号
stat file.txt   # 查看完整 inode 信息
```

1.4.3 inode 与文件删除的关系 [p.14] (重要考点)

- **rm** 删除文件，磁盘空间不一定立刻释放
- 删除文件实际做两件事：
 1. 目录中删除” 文件名 → inode” 映射
 2. inode 硬链接计数减 1
- 只有链接计数降为 0，且没有进程打开该文件时，inode 和数据块才被标记为空闲
- 文件被进程占用时 rm 删除，空间不会立刻释放，进程退出/关闭文件后才回收

1.4.4 inode 限制 [p.15]

inode 数量在格式化时确定。**inode 用尽时即使磁盘有空间也无法创建新文件。**

```
dumpe2fs -h /dev/sda1 | grep -i inode
```

1.5 链接机制 [p.16–21]

1.5.1 硬链接 vs 软链接 [p.16]

特性	硬链接	软链接（符号链接）
inode	同 inode, <code>ls -li</code> 编号一致	拥有独立 inode
链接计数	<code>Links > 1</code>	独立计数
删除原文件	硬链接依旧可用	软链接失效（悬空链接）
跨文件系统	不可以	可以
链接目录	不可以（防循环引用）	可以
权限	同原文件	通常 777，实际由目标决定
创建命令	<code>ln target link</code>	<code>ln -s target link</code>

1.5.2 硬链接应用场景 [p.20]

```
ln /data/important.db /backup/important.db.backup # 文件备份（不占额外空间）
ln important.txt important.txt.backup             # 防止误删
```

1.5.3 软链接应用场景 [p.21]

```
ln -s /opt/python-3.9.0/bin/python3 /usr/local/bin/python # 全局快捷入口
ln -sf /opt/app-v2.2.0 /opt/app-current                   # 版本切换
ln -s /data/logs /var/log/myapp                           # 日志目录重定向
ln -s /mnt/storage/shared /home/shared                   # 跨分区引用
```

2 权限模型基础 [p.23–31]

2.1 UGO 权限模型 [p.23]

Linux 采用 **UGO** 模型：

- **User**（所有者）：文件创建者或指定用户
- **Group**（所属组）：关联的用户组
- **Others**（其他用户）：除所有者、组成员外所有用户

9 个权限字符：**rw-rw-rw-**，3 个一组，分别代表 U/G/O 的读/写/执行。

2.2 权限数字表达 [p.24]

字符	八进制	计算
—	0	0+0+0
-x	1	0+0+1
-w-	2	0+2+0
-wx	3	0+2+1
r-	4	4+0+0
r-x	5	4+0+1
rw-	6	4+2+0
rwX	7	4+2+1

常用标准权限：644（普通文件）、755（目录/可执行）、600（私钥/配置）、700（私密目录）、400（只读）。

2.3 权限对文件和目录的不同含义（核心考点） [p.25–26]

权限	对文件	对目录
r (4)	读取内容 (cat, less, grep)	ls 列出目录内容
w (2)	修改/写入内容 (vim, echo »)	创建、删除、重命名文件
x (1)	执行脚本/二进制程序	cd 进入目录，访问文件元数据 (stat)

经典易错点：

- 对文件有 w 权限不代表能删除文件——删除需要对所在**目录**有 w 权限！
- 即使对文件有 rwx，若对上级目录无 x 权限，依旧无法访问——路径上任何目录缺少 x 都会导致访问失败

2.4 权限命令 [p.27–28]

2.4.1 chmod [p.27]

```
chmod 644 file.txt      # 数字方式：rw-r--r--
chmod u+x file.sh       # 符号方式：给所有者增加执行权限
```

```

chmod g-w file.txt      # 给组移除写权限
chmod o=r file.txt      # 其他用户只读
chmod a+r file.txt      # 所有用户增加读权限
chmod -R 755 /var/www   # 递归修改目录所有文件权限

```

符号语法: `chmod [u/g/o/a] [+/-/=] [r/w/x] file`

2.4.2 chown / chgrp [p.28]

```

chown user file.txt      # 修改所有者
chown user:group file.txt # 修改所有者和组
chown :group file.txt    # 仅修改组
chown -R user:group /data # 递归修改
chgrp group file.txt     # 通过 chgrp 修改组

```

2.5 移动/重命名与目录权限 [p.30]

移动文件需要同时满足：对源目录有 **w** 权限且对目标目录有 **w** 权限。因为 `mv` 本质上是在目标目录创建新目录项 + 原目录删除旧目录项。

3 umask 默认权限 [p.32–34]

3.1 umask 原理 [p.32]

`umask` 控制新建文件和目录的默认权限，从系统默认权限中“减去”相应位：

$$\text{最终权限} = \text{默认权限} - \text{umask}$$

- 新建文件默认：666 (rw-rw-rw-) —— 默认无执行权限（安全原因）
- 新建目录默认：777 (rwxrwxrwx) —— 目录需要 **x** 权限才能进入

3.2 常用 umask [p.33]

umask	文件默认	目录默认	说明
000	666	777	极不安全
002	664	775	组可写（协作常用）
022	644	755	大多数发行版默认
027	640	750	其他用户无权限
077	600	700	仅所有者（最安全）

```

umask      # 查看当前 umask（八进制）
umask -S   # 查看符号形式

```

```
umask 027          # 当前会话设置
echo "umask 027" >> ~/.bashrc # 永久生效
```

最佳实践：在脚本中不应依赖 `umask`，应显式设置权限（`chmod 600 config.txt`）。

4 特殊权限 [p.35–43]

4.1 三大特殊权限总结 [p.42]

权限	八进制	作用对象	标识位
SUID	4	可执行文件	所有者 x 位 → s/S
SGID	2	文件/目录	组 x 位 → s/S
Sticky Bit	1	目录	其他 x 位 → t/T

4.2 SUID (Set User ID) [p.35–37]

作用：执行时临时拥有文件所有者权限。典型应用：`/usr/bin/passwd`（普通用户修改自己的密码需临时获得 root 权限写 `/etc/shadow`）。

```
chmod 4755 file      # 数字方式设置 SUID
chmod u+s file       # 符号方式设置
chmod u-s file       # 取消 SUID
```

安全规范 [p.37]:

- 尽量少用 SUID，确保程序来源可信
- 定期检查：`find / -perm -4000 -type f 2>/dev/null`
- 绝对不要对 Shell 脚本设置 SUID
- 优先使用 `sudo` 代替 SUID

4.3 SGID (Set Group ID) [p.38–39]

对可执行文件：运行时继承文件所属组权限。

对目录：目录内新建文件/子目录自动继承目录所属组。典型应用：团队共享目录。

```
chmod 2775 /project    # SGID + rwxrwxr-x
chmod g+s /shared      # 符号方式
# 团队共享目录：新建文件自动属于 developers 组
```

4.4 Sticky Bit (SBIT) [p.40–41]

作用：公共目录中，仅文件所有者、目录所有者、root 可删除/重命名自己的文件。典型应用：`/tmp`。

```
chmod 1777 /tmp        # Sticky + rwxrwxrwx
chmod +t /tmp          # 符号方式
```

4.5 综合范例 [p.43]

```
# 团队共享项目目录: SGID(文件继承组) + Sticky(防止互删) + 770
mkdir -p /data/project
chown root:project /data/project
chmod 3770 /data/project      # 3 = SGID(2) + Sticky(1)
```

5 ACL 访问控制列表 [p.44–46]

目的: 突破 UGO 三种身份的局限, 为多个特定用户/组设置不同权限 [p.44]。

核心概念:

- ACE (Access Control Entry): 格式为” 类型: 标识符: 权限”, 类型为 u(用户)/g(组)/m(掩码)
- 掩码 (mask): 限制其他所有 ACL 条目的最大有效权限

<code>getfacl file.txt</code>	# 查看 ACL
<code>setfacl -m u:alice:rw project.txt</code>	# alice 有读写
<code>setfacl -m g:developers:rw /project</code>	# developers 组有读写
<code>setfacl -x u:alice file.txt</code>	# 删除 alice 的 ACL
<code>setfacl -b file.txt</code>	# 清除所有 ACL
<code>getfacl file1.txt setfacl -M - file2.txt</code>	# 复制 ACL

注意: 当文件设置了 ACL 时, `ls -l` 权限位末尾显示 + 号。